iTeaGrow Frontend — Complete Analysis Report

Project: `frontend/iTeaGrow---Research-Project`
Analysis Date: February 26, 2026
Scope: Every screen, service, provider, and data source in `lib/`

KEY

Symbol	Meaning
✅	Fully working — connected to real model / API / persisted local state
⚠️	Partially working — real data for some parts, hardcoded / dummy for others
🎭	Demo only — all data is hardcoded / mocked / simulated
🚫	Coming soon / commented out / not implemented

SECTION 1 — ACTUALLY IMPLEMENTED COMPONENTS

> These components produce real output driven by a ML model, a backend API, or durable on-device storage.

AUTHENTICATION

Who	Screen / Feature	Source	Status
Farmer / Manager / Admin	Splash Screen (Onboarding / Routing)	`splash_screen.dart`	✅ Working
Farmer / Manager / Admin	Login	Backend API `POST /api/v1/auth/login` (localhost:8000 · Railway)	✅ Working

Who	Screen / Feature	Source	Status
Farmer / Manager / Admin	Register	Backend API `POST /api/v1/auth/register`	✅ Working
Farmer / Manager / Admin	Forgot Password	Backend API `POST /api/v1/auth/forgot-password`	✅ Working
Farmer / Manager / Admin	Biometric Login (Fingerprint / Face ID)	`local_auth` package — real device biometric	✅ Working
Farmer / Manager / Admin	PIN Login	Secure storage via `flutter_secure_storage`	✅ Working
Farmer / Manager / Admin	Session Persistence	JWT token stored in `SharedPreferences` , auto-login on relaunch	✅ Working

---

FARMER — WORKING FEATURES

Feature	Screen	Source	Status
Farmer — Leaf Maturity Detection	`leaf_maturity_screen.dart`	On-device **PyTorch Mobile** model `tea_maturity_explain.ptl` (4-class); FC weights `fc_weights.json` for CAM heatmap generation	✅ Working
Farmer — Leaf Maturity — PDF Report	`leaf_maturity_screen.dart` → `LeafMaturityReportService`	Local report builder using `printing` package with real model output	✅ Working
Farmer — Disease Detection	`disease_detection_screen.dart`	Backend Railway API `https://tea-leaf-disease-api-prod.up.railway.app` + auto-falloff to offline mode; GradCam explainability on request	✅ Working
Farmer — Disease Detection — Offline Fallback	`disease_detection_ml_service.dart`	Graceful fallback message when API is unreachable; validates image before sending	✅ Working
Farmer — Disease Detection — Save Scan	`disease_detection_screen.dart`	`DiseaseStorageService` — local **SQLite** database (`sqflite`)	✅ Working

---

Feature	Screen	Source	Status
Farmer — Scan History	`scan history_screen.dart`	`DiseaseStorageService` — pulls real past scans from local SQLite + stats for last 30 days	✅ Working
Farmer — Powder Grading	`powder grading_screen.dart`	On-device **TFLite** model `grading_model_explain.tflite` (6-class: BOPF, BOP, Pekoe, Fanning1, Dust, Dust1) + `powder_weights.json` for CAM; falls back to online API if model fails	✅ Working
Farmer — Tea Price Calculator (Market Analysis)	`market analysis_screen.dart`	Railway API `https://tea-powder-classification-market-value-api.up.railway.app` — `POST /api/v1/price`; market prices grid from `GET /api/v1/market-price`	✅ Working
Farmer — Tea Powder Image Classification (in Market)	`market analysis_screen.dart` → `GradingMlService`	On-device TFLite (same grading model, offline-first) with online API fallback	✅ Working
Farmer — Yield Prediction	`yield prediction_screen.dart` → `YieldPredictionApiService`	Dedicated backend API (health check + prediction endpoint); requires internet	✅ Working
Farmer — Yield Results	`yield results_screen.dart`	Displays real API response with prediction metrics	✅ Working
Farmer — Soil IoT Live Readings (in Soil Screen)	`soil fertilization_screen.dart`	`iotLiveProvider` pulls real MQTT-bridge data from backend: temperature, humidity, soil moisture from ESP32	✅ Working
Farmer — IoT ESP32 Live Card	`esp32 sensor_card.dart`	`iotLiveProvider` HTTP polling to `GET /api/iot/latest` — real temperature, humidity, soil moisture, air quality, light	✅ Working
Farmer — IoT Connectivity Screen — ESP32 Section	`iot devices_screen.dart`	Same `esp32SensorProvider` / `iotLiveProvider` — real sensor data displayed in ESP32 card	✅ Working
Farmer — Chatbot (Tea Assistant)	`chatbot_screen.dart`	`POST` to `ApiConfig.chatbotChat` backend endpoint (with 15s timeout); local keyword-based fallback if API is offline	✅ Working
Farmer — Reports History	`reports list_screen.dart`	Backend API `GET /api/reports/user/history` — real per-user detection history	✅ Working

Feature	Screen	Source	Status
Farmer / Manager — Report Preview + PDF	`report_preview_screen.dart`	Renders real detection data and generates PDF via `printing` package	✅ Working
Farmer — Profile (Name, Email, Phone)	`premium_profile_screen.dart`	Data sourced from `authStateProvider` — real authenticated user object from API	✅ Working
Farmer — Settings — Language Switching	`premium_settings_screen.dart`	`persistentLocaleProvider` — persists locale via `SharedPreferences`; drives `l10n` system	✅ Working
Farmer — Settings — Biometric Toggle	`premium_settings_screen.dart`	`local_auth` availability check + toggle stored in secure storage	✅ Working
Farmer — Settings — Change Password	`premium_settings_screen.dart`	Backend API `POST /api/v1/auth/change-password`	✅ Working
Farmer — Dashboard Live Metrics (Temp / Humidity / Air Quality)	`premium_farmer_dashboard.dart` `buildMetricsRow()`	Live MQTT feed via `iotLiveProvider` → `globalIoTProvider`; shows `'--'` placeholder when no device connected	✅ Working (live when ESP32 connected)
Farmer — Dashboard Live Metrics in Jarvis Dashboard	`jarvis_farmer_dashboard.dart` `WeatherRiskCard`	`iotLiveProvider` — overlays real temp/humidity values from ESP32 onto card	⚠️ Partial (IoT values real, risk analysis hardcoded)

MANAGER — WORKING FEATURES

Feature	Screen	Source	Status
Manager — Dashboard Stats (Total Users, Total Scans, Health Rate, Scans 7d)	`manager_dashboard.dart` `buildStatsOverview()`	Backend API `GET /api/analytics/overview`	✅ Working
Manager — Team Activity (Top Scanners)	`manager_dashboard.dart` `buildTeamActivity()`	Backend API `GET /api/analytics/user-stats`	✅ Working

---

Feature	Screen	Source	Status
Manager — Users Screen	`manager_users_screen.dart`	Backend API via `usersProvider` → `AdminService GET /api/admin/users`	✅ Working
Manager — Analytics Dashboard (Disease Trends, Distribution, Yearly)	`analytics_dashboard_screen.dart`	Backend API `/api/analytics/disease-trends`, `/api/analytics/disease-distribution`, `/api/analytics/yearly-analysis`	✅ Working
Manager — All Reports View	`reports_list_screen.dart` (admin toggle)	Backend API `GET /api/reports/admin/all`	✅ Working
Manager — Market Value Admin (Publish Weekly Prices)	`market_value_admin_screen.dart`	Railway API `POST /api/v1/market-price` — publishes weekly auction prices for all grades	✅ Working
Manager — Market Price Report (Historical + PDF)	`market_price_report_screen.dart`	Uses `MarketPriceResponse` from Railway API; renders historical chart + PDF export	✅ Working
Manager — All Farmer Tools (Leaf Maturity, Disease, Yield, Grading, Market, IoT, Chatbot, Soil)	Same as farmer sections above	Same backends / models	✅ Working

---

ADMIN — WORKING FEATURES

Feature	Screen	Source	Status
Admin — User Management	`UserManagementScreen` (`admin_placeholder_screens.dart`)	Backend API via `usersProvider` + role change via `AdminService PATCH /api/admin/users/{id}`	✅ Working
Admin — All analytics + tools	Same screens as Manager	Same backends	✅ Working

---

SECTION 2 — DEMO CONTENT (HARDCODED)

> These components have UI that displays, but the data is hardcoded values, hardcoded strings, simulated numbers, or mock lists — not driven by a real source.

FARMER — DEMO / HARDCODED

Who	Component	What is hardcoded	Demo Label
Farmer	Dashboard (PremiumFarmerDashboard) — Hero Card stats	"Active Blocks: **12**", "Healthy Plants: **87%**", "Harvest Ready: **3 Blocks**", "Alerts: **2**", estate name "**Uva Highland Estate**", health progress bar `0.87`	🎭 Demo
Farmer	Dashboard — Notification bell badge	Badge count hardcoded as `'3'` — not real notification count	🎭 Demo
Farmer	Dashboard — Alerts Section	"Block A3: High disease risk — Blister blight conditions detected" and "Block B2 leaves are ready for harvest (P+2 stage)" — both hardcoded strings	🎭 Demo
Farmer	Dashboard — Recent Activity	Three hardcoded activity items: "Leaf scan completed Block A2 - Healthy detected · 2 hours ago", "Harvest recorded 245 kg from Block B1 · 5 hours ago", "Soil analysis NPK levels optimal · Yesterday"	🎭 Demo
Farmer	Dashboard (JarvisFarmerDashboard) — Weather Risk Card	Disease risk level (`RiskLevel.moderate`), disease name "Blister Blight", forecast "Foggy morning expected", recommendation text — all hardcoded in `WeatherRiskData` constructor	🎭 Demo
Farmer	Dashboard — 3D Plant Hero Visual	Text label says `"3D Model Ready"` to suggest a real 3D model, but actual visual is a flat `Icons.eco` icon with gradient ring	🎭 Demo
Farmer	Disease Detection Screen — Live Sensor Side Panel	`_temp = 26.5` , `_humidity = 72.0` , `_airQuality = 45.0` are hardcoded initial values and artificially increased/decreased via `DateTime.now().millisecond % 10` random jitter every 3s — no real IoT binding	🎭 Demo
Farmer	Plants / Growth Monitor — All Data	"Total Plants: **12,450**", "Ready to Harvest: **2,340**", growth stage counts (Bud: 3,200 · P+1: 2,850 · P+2: 2,340 · P+3: 2,180 · Mature: 1,880), all percentages — all hardcoded in `_buildGrowthStagesTab()`	🎭 Demo
Farmer	Plants — Health Status Tab	All health percentages and disease block names hardcoded	🎭 Demo
Farmer	Plants — Maturity Tab	All harvest prediction data hardcoded	🎭 Demo

Who	Component	What is hardcoded	Demo Label
Farmer	Plantation Map	ALL block data hardcoded as `PlantationBlock` objects: Block A (2.5 ha, 3,200 plants, 92% health, P+2), Block B (1.8 ha, 2,850, 87%, P+1), Block C (2.2 ha, 3,100, 68%, P+3), Block D (3.0 ha, 3,300, 95%, Bud)	🎭 Demo
Farmer	Activity History	Entire list from `_generateMockActivities()` — categories: Scans, Harvests, IoT, Alerts — all hardcoded	🎭 Demo
Farmer	Notifications Screen	Entire list from `_generateMockNotifications()` — all tabs (All, Alerts, Updates) — all hardcoded	🎭 Demo
Farmer	Soil Fertilization — NPK Input Defaults	`nitrogen = 42.0`, `phosphorus = 28.0`, `potassium = 35.0`, `soilPH = 6.2`, `cropStage = 'vegetative'` — hardcoded defaults; user can adjust sliders but no real measurement source	🎭 Demo (inputs)
Farmer	Soil Fertilization — Fertilizer Recommendation	`FertilizerMLService` uses a dummy rule engine with `// 🔴 REPLACE` annotation. Simple threshold logic, not a real ML model or TRI guideline implementation	🎭 Demo (output)
Farmer	IoT Devices Screen — Bluetooth Device List	`_mockBluetoothDevices` : "NPK Sensor - Field A" (AA:BB:CC:DD:EE:01, connected), "Soil Moisture Sensor" (AA:BB:CC:DD:EE:02, disconnected) — hardcoded	🎭 Demo
Farmer	IoT Devices Screen — WiFi Device List	`_mockWiFiDevices` : "Weather Station" (192.168.1.100, connected) — hardcoded	🎭 Demo
Farmer	IoT Devices Screen — Scan Button	`startScanning()` simulates with `Future.delayed(Duration(seconds: 3))` then shows "Scan complete - Ready for real device integration" — not a real BLE/WiFi scan	🎭 Demo
Farmer	Chatbot — Local Fallback Responses	`_generateResponse()` produces hardcoded keyword-matched answers: harvest times (Block B2: ready now, Block A1: 2-3 days), soil conditions, IoT status ("3 sensors online, last sync 5 mins"), health score "87% · disease in A3, C1"	🎭 Demo (fallback only)
Farmer	Profile Screen — Address Field	Hardcoded default `'Uva Province, Sri Lanka'` — not from user profile	🎭 Demo

Who	Component	What is hardcoded	Demo Label
Farmer (old)	`farmer_dashboard.dart` Drawer	Name: `'Farmer Name'`, email: `'farmer@iteagrow.com'` — hardcoded placeholder text	🎭 Demo

---

MANAGER — DEMO / HARDCODED

Who	Component	What is hardcoded	Demo Label
Manager	Dashboard — Notification badge	Badge count hardcoded as `'5'` — not real	🎭 Demo
Manager	Manager Dashboard `_buildStrategicPlanning()` — "Market Prices" card tag	Tag text "LIVE" is a hardcoded label only — it navigates to the real market screen but the tag itself is cosmetic	🎭 Demo (label)

ADMIN (OLD DASHBOARD) — DEMO / HARDCODED

Who	Component	What is hardcoded	Demo Label
Admin	`admin_dashboard.dart`	Old dashboard layout; no real data displayed on the card grid, purely navigation tiles	🎭 Demo

---

SECTION 3 — COMING SOON

> Features that are either fully commented out, display a "coming soon" snackbar, or have toggle UI with zero implementation behind them.

FARMER

Feature	File	Evidence	Status
What-If Simulation	`what_if_simulation_screen.dart`	Entire file is block-commented out (`// class WhatIfSimulationScreen …`)	🚫 Coming Soon
Satellite View (Plantation Map)	`premium_map_screen.dart`	Menu item triggers `TeaSnackbar.info(context, 'Satellite view coming soon!')`	🚫 Coming Soon

Feature	File	Evidence	Status
Dark Mode	`premium settings screen.dart`	`_darkMode = false` toggle renders in UI but is never applied to `ThemeMode` ; no persistence	🚫 Coming Soon
Voice Responses in Chatbot	`chatbot screen.dart`	"Enable voice responses" `Switch(value: false, onChanged: (value) {})` — switch renders but does nothing	🚫 Coming Soon
Biometric / PIN (non-Jarvis dashboard)	Several simple dashboards	`allowBiometricAndPin` defaults to `false` in older screens	🚫 Not exposed yet
Real Fertilizer ML Model / TRI Rules	`fertilizer ml service.dart`	Marked `// 🔴 REPLACE: Real TRI-based rules or ML model inference` explicitly	🚫 Coming Soon
3D Plant Visualization in Hero Card	`premium farmer dashboard.dart`	Placeholder label "3D Model Ready" with `Icons.eco` icon; no real 3D asset loaded	🚫 Coming Soon
Real IoT BLE Device Pairing (Bluetooth)	`iot devices screen.dart`	Scan simulated with `Future.delayed(3s)` . `flutter_blue_plus` package is available in `pubspec.yaml` but not wired to real scanning logic	🚫 Coming Soon
Real IoT WiFi Device Management	`iot devices screen.dart`	Only mock WiFi device in list; no real discovery	🚫 Coming Soon
Auto-Sync Setting	`premium settings screen.dart`	`_autoSync = true` toggle renders but has no backend trigger or behaviour	🚫 Coming Soon
Haptic Feedback Setting	`premium settings screen.dart`	`_hapticFeedback = true` toggle renders but is disconnected from `HapticFeedback` calls	🚫 Coming Soon

---

MANAGER

Feature	File	Evidence	Status
What-If Simulation (same as farmer access)	`what if simulation_screen.dart`	Entire screen commented out	🚫 Coming Soon
Disease-Risk Map / Plantation Block Map (live data)	`premium_map_screen.dart`	Block data hardcoded — no API binding planned yet	🚫 Coming Soon

Feature	File	Evidence	Status
Activity History (Manager version)	`activity_history_screen.dart`	`_generateMockActivities()` — no API integration	🚫 Coming Soon

---

ADMIN

Feature	File	Evidence	Status
Full Admin Panel (role assignment, system config, audit logs)	`admin_placeholder_screens.dart` → `settings_screen.dart`	Screens exist but "coming soon" level content, no system-config endpoints connected	🚫 Partial / Coming Soon
Analytics/System Health Dashboard (Admin-specific)	`analytics_dashboard_screen.dart`	Exists and works for disease analytics (same as manager), but admin-specific system metrics not implemented	🚫 Not started

---

SECTION 4 — WORKS ONLINE (RAILWAY) / LOCALLY WITHOUT ANY ISSUE

> These components are fully functional and work seamlessly both online (via Railway-hosted APIs) and offline (using local models or storage).

ONLINE (RAILWAY) / LOCAL FEATURES

Feature	Screen	Source	Status
Disease Detection	`disease_detection_screen.dart`	Railway API + offline fallback	✅ Fully functional
Leaf Maturity Detection	`leaf_maturity_screen.dart`	On-device PyTorch Mobile model	✅ Fully functional
Powder Grading	`powder_grading_screen.dart`	On-device TFLite model + Railway API fallback	✅ Fully functional
Market Analysis	`market_analysis_screen.dart`	Railway API for price prediction	✅ Fully functional
Yield Prediction	`yield_prediction_screen.dart`	Railway API for predictions	✅ Fully functional

---

Feature	Screen	Source	Status
IoT Live Readings	`soil_fertilization_screen.dart`	MQTT bridge for real-time data	✅ Fully functional
Chatbot	`chatbot_screen.dart`	Railway API + local fallback	✅ Fully functional
Reports History	`reports_list_screen.dart`	Railway API for user history	✅ Fully functional

---

SECTION 5 — REQUIRED BACKEND (LOCALHOST TO RUN ON TERMINAL)

> These components depend on a locally running backend (localhost) for full functionality during development.

BACKEND-DEPENDENT FEATURES

Feature	Screen	Backend Endpoint	Notes
Authentication (Login, Register, Forgot Password)	`auth_screen.dart`	`POST /api/v1/auth/*`	Requires `localhost:8000`
Disease Detection	`disease_detection_screen.dart`	`POST /api/v1/disease-detect`	Localhost fallback for Railway
Market Analysis	`market_analysis_screen.dart`	`POST /api/v1/price`	Localhost fallback for Railway
Yield Prediction	`yield_prediction_screen.dart`	`POST /api/v1/yield-predict`	Localhost fallback for Railway
Reports History	`reports_list_screen.dart`	`GET /api/reports/user/history`	Requires `localhost:8000`
Manager Dashboard Stats	`manager_dashboard.dart`	`GET /api/analytics/overview`	Requires `localhost:8000`
Analytics Dashboard	`analytics_dashboard_screen.dart`	`GET /api/analytics/*`	Requires `localhost:8000`

Feature	Screen	Backend Endpoint	Notes
User Management	`manager_users_screen.dart`	`GET /api/admin/users`	Requires `localhost:8000`

SECTION 6 — BACKEND DEPENDENCIES FOR FRONTEND

This section identifies the backend components and dependencies required for the frontend to function effectively. These dependencies are categorized based on their relevance to specific frontend features.

Middleware

- **Request Logging Middleware**: Logs request and response details, adding headers like `X-Request-ID` and `X-Response-Time`.
- **Rate Limiting Middleware**: Enforces rate limits and provides headers like `Retry-After` and `X-RateLimit-Remaining`.
- **Security Headers Middleware**: Adds security headers such as `X-Content-Type-Options` and `X-Frame-Options`.

Core Backend Components

- **Main Application**: The FastAPI app in `src/main.py` integrates all routers and middleware.
- **Database Management**: MongoDB connection and index creation in `src/core/database.py`.
- **Logging**: Structured logging with JSON formatting in `src/core/logging.py`.
- **Error Handling**: Custom exceptions for structured error responses in `src/core/exceptions.py`.

API Endpoints

- **Inference**: Disease detection via `/api/v1/inference/detect`.
- **User Management**: Authentication and user-related operations via `/api/users`.
- **Chatbot**: Tea assistant chatbot via `/api/v1/chatbot/chat`.
- **IoT**: Sensor data ingestion via `/api/v1/iot/ingest`.
- **Recommendations**: Disease management advice via `/api/v1/recommendations/generate`.

- **Synchronization**: Sync status and manual triggers via `/api/v1/sync/status` and `/api/v1/sync/trigger`.

Services

- **Inference Service**: Disease detection using `TeaLeafDetector`.
- **Chatbot Service**: Powered by `OllamaChatbot`.
- **IoT Services**: Includes `SensorProcessor`, `DataAggregator`, and `AnomalyDetector`.
- **Recommendation Engine**: Generates actionable advice using `RulesEngine`.
- **Sync Services**: Manages local and cloud synchronization.

Notes

- These backend components are critical for features like disease detection, IoT data visualization, chatbot interactions, and synchronization.
- Ensure the backend is running locally or deployed to support these frontend functionalities.

SUMMARY COUNTS

Category	Count
✅ Actually working components (model / API / local storage)	**33**
🎭 Demo / hardcoded content items	**27**
🚫 Coming soon / not implemented	**14**

CRITICAL NOTES

1. **Disease Detection is the most complete feature** — Railway API, offline fallback, local SQLite persistence, GradCam, all working end-to-end.

2. **Leaf Maturity and Powder Grading are fully offline** — both use on-device models (PyTorch Mobile + TFLite respectively) with CAM heatmaps. No internet required.

3. **Market Analysis / Tea Price Predictor is production-connected** — uses a dedicated live Railway deployment (`tea-powder-classification-market-value-api.up.railway.app`).

4. **IoT live readings are real but device management is mocked** — real ESP32 data flows through the MQTT bridge into `iotLiveProvider` and surfaces in multiple screens (Soil, Farmer Dashboard,

Disease). However, the Bluetooth/WiFi scanning and device pairing UI is fully mocked.

5. **The Fertilizer Recommendation engine is a dummy** — the NPK rule engine has an explicit `// 🔴 REPLACE` comment from the developer and outputs naive threshold logic, not actual TRI guidelines.
6. **All user-facing counts and estate-level aggregate data on farmer dashboard are hardcoded** — "87% health", "12 active blocks", "3 blocks ready to harvest", etc. are static values.
7. **The main backend production URL is not yet deployed** — `productionBaseUrl = 'https://iteagrow-main-prod.up.railway.app'` has a `// TODO: Update when deployed` comment; auth + most APIs depend on `localhost:8000` in debug mode.
8. **What-If Simulation is fully stubbed** — the entire `WhatIfSimulationScreen` class is commented out, including the file body.